

ARM EXAM SOLUTIONS

4 July 2023

Aliquot sums, sociable numbers, Timer 1 and LED output

Complete worked answers to Questions 1 and 2

LPC1768 / Cortex-M3 | Keil ARMASM and C

Included: step-by-step reasoning, register and stack explanations, all expected sequences, timer calculations, LED mapping, project instructions, and the full source code.

Verified: four native compile/link builds; native instruction execution against mathematical references; boundary and compiled interrupt-handler checks. Physical board timing has not been tested.

Open Q1_Assembly/sample.uvprojx for Question 1.

Open Q2_Timer_LEDs/sample.uvprojx for the complete application.

1. What we are building

The source is the two-page paper **20230704.pdf**, Computer Architectures, part II. Question 1 asks for an ARM assembly function named `isSociable`. Question 2 uses that function inside a Timer 1 interrupt handler. This solution targets the LPC1768 / Cortex-M3 using the supplied Keil project and its existing `exam_api` helpers.

There are two complete projects. **Q1_Assembly** runs the paper's four examples so you can inspect the answers. **Q2_Timer_LEDs** extends the same assembly with the periodic LED application. The assembly answer is identical in both projects.

The final answers for the seven values in Question 2 are **1, 2, 0, 5, 4, 2, 4**. The corresponding physical LEDs are **LD4, LD5, all off, LD8, LD7, LD5, LD7**.

2. Understand aliquot sums before writing assembly

A divisor of n divides n exactly, leaving remainder zero. A proper divisor excludes n itself. The aliquot sum $s(n)$ adds all those proper divisors.

For 28, the proper divisors are 1, 2, 4, 7, and 14. Therefore $s(28) = 28$. This makes 28 a perfect number: one application of s returns to the starting number.

For 220, $s(220) = 284$ and $s(284) = 220$. It takes two applications to return to the start, so 220 and 284 form an amicable pair.

For 12496, the chain is:

```
12496 -> 14288 -> 15472 -> 14536 -> 14264 -> 12496
      1       2       3       4       5
```

The numbers below the arrows' destinations count computed aliquot sums. The starting input does not count as a computed term. The answer is 5, not 6.

The exam groups all these cases under `isSociable`: result 1 means perfect, result 2 means amicable, and results 3 through 8 represent a longer cycle found within the allowed search. A result of 0 means the function did not establish a return to the starting number under the exam's stopping rules. Reaching eight terms alone does not prove that a longer cycle is impossible.

3. Translate the stopping rules precisely

Keep three values: the original input, the current sequence value, and the number of computed sums. Initially, `original = current = n` and `count = 0`.

Each iteration computes exactly one sum. After that calculation, use this order:

1. Increment count.
2. If the new sum equals the original input, return count.
3. If the new sum is 1, return 0.
4. If count is 8, return 0.
5. Otherwise, make the new sum the current value and repeat.

Checking success before the eight-term limit is essential. A number that returns to its start on the eighth computation must return 8. Checking the limit first would wrongly reject it.

Compare against the original input, not the previous term. A chain can reach a cycle that does not contain its starting number. That does not make the starting number sociable. For example, 25 produces 6, then 6 repeatedly; because it never returns to 25, this routine returns 0 after eight computations.

For 100, the trace is 100 -> 117 -> 65 -> 19 -> 1. The routine returns 0 when it reaches 1, after four sums.

The paper's divisor algorithm assumes $n \geq 2$. The delivered code additionally returns 0 immediately for input 0 or 1, and defines `aliquotSum(0)` and `aliquotSum(1)` as 0 for safe execution. These guards do not change any of the exam's examples. The paper does not specify arithmetic-overflow handling; this implementation assumes the input and every computed sum fit an unsigned 32-bit integer, as the supplied cases do.

4. The divisor-pair algorithm required by the paper

The paper explicitly supplies an algorithm. The assembly follows its order, including the rule that the exit comparison happens only after an exact divisor is found.

Initialize `sum = 1` because 1 is a proper divisor of every $n \geq 2$. Initialize `a = 2` because divisor 1 is already included. We never start with `sum = n + 1`: n itself is excluded.

For each candidate a , calculate the integer quotient $b = n / a$ and the remainder $n - b \cdot a$. A zero remainder means a divides n exactly. When it does:

- If $a < b$, both a and b are distinct proper divisors. Add both.
- If $a == b$, n is a perfect square at this pair. Add a once and finish.
- If $a > b$, this pair is the reverse of a pair already considered. Finish without adding it.

If there is no exact division, increment a . After adding a distinct pair, also increment a .

Worked trace: `s(28)`

Candidate a	Quotient b	Remainder	Action	Sum
Initial	-	-	Start with divisor 1	1
2	14	0	Add 2 and 14	17
3	9	1	Skip	17
4	7	0	Add 4 and 7	28
5	5	3	Skip: division is not exact	28
6	4	4	Skip: division is not exact	28
7	4	0	$7 > 4$, stop	28

At $a = 5$, the integer quotient happens to equal a , but that is not enough to call it a square. The remainder is nonzero. This is why the divisibility test must precede the a/b comparison in the supplied algorithm.

For a square such as 16, start at 1, add 2 + 8, skip 3, then add 4 once. The result is 15. Adding 4 twice would incorrectly produce 19.

For a prime such as 7, there is no divisor from 2 through 6. At $a = 7$, the quotient is 1 and the remainder is zero, so $a > b$ ends the loop, returning 1. This literal algorithm can scan all the way to n for primes. It is not an $O(\sqrt{n})$ implementation. An earlier square-root cutoff could improve runtime, but the submitted answer retains the algorithm requested by this paper.

5. C and assembly agree on the function interface

The C declaration is:

```
extern uint32_t isSociable(uint32_t n);
```

There is one unsigned 32-bit input and one unsigned 32-bit return value. C puts the input in R0 before calling the assembly function. The assembly puts its answer in R0 before returning.

EXPORT isSociable makes the assembly symbol visible to the linker. The spelling and capitalization must exactly match the C declaration. THUMB selects the instruction mode used by Cortex-M3. AREA declares a read-only code section. PROC and ENDP mark a procedure in ARMASM source. PRESERVE8 declares that the code maintains eight-byte stack alignment; the actual PUSH/POP choices must uphold that declaration.

Register plan for isSociable

Register	Meaning	Why it is chosen
R0	Argument/result of aliquotSum	Standard input and return register
R4	Original n	Must survive every helper call
R5	Current sequence value	Must survive helper calls
R6	Number of computed sums	Must survive helper calls
LR	Return address	BL overwrites it, so save it

R0-R3 and R12 may be changed by a called function. R4-R11 are preserved by a function that uses them. Keeping persistent values in R4-R6 is therefore appropriate, but our function must save and restore their incoming values.

PUSH {R4-R6, LR} saves four 32-bit registers: 16 bytes. An eight-byte-aligned incoming stack remains aligned before BL aliquotSum. POP {R4-R6, PC} restores the saved registers and returns by loading the saved address into the program counter.

The helper uses only R0-R3 and R12 and makes no nested calls. It can return with BX LR without a PUSH/POP pair. The caller's original LR remains safe on the stack while BL temporarily writes the helper return address into LR.

6. Read the assembly one block at a time

The full copyable source is Source/ASM_func.t.s in each project. Appendix A contains the same complete file.

Starting isSociable

After saving registers, the function rejects inputs below 2. `MOV R4, R0` preserves the original input, `MOV R5, R0` sets the first current value, and `MOVS R6, #0` initializes the count.

At `sequence_loop`, `MOV R0, R5` passes the current value to the helper. `BL aliquotSum` calls it. The returned `R0` is the next term in the chain. `ADDS R6, R6, #1` counts that computation.

`CMP R0, R4` compares the sum with the original `n`. `BEQ found` branches on equality. At `found`, moving `R6` into `R0` returns the cycle length.

If it did not return to the start, `CMP R0, #1` and `BLS not_found` reject 0 or 1 using an unsigned comparison. The paper asks for the 1 case; accepting 0 as another failure case is the defensive extension described earlier. The next comparison rejects count 8. Otherwise, `MOV R5, R0` advances the sequence.

Calculating division and remainder

```
UDIV R3, R0, R2      ; b = unsigned n / a
MLS  R12, R3, R2, R0 ; remainder = n - b*a
```

`UDIV` performs integer unsigned division. It does not also return the remainder. `MLS` multiplies the middle two operands and subtracts that product from the last operand. Here it calculates `R0 - R3*R2` and places the result in `R12`. For `n = 28` and `a = 3`, `b = 9` and the remainder is `28 - 9*3 = 1`.

`CMP R12, #0` explicitly sets the condition flags for the remainder check. `BNE next_divisor` skips non-divisors. For an exact divisor, `CMP R2, R3` compares `a` against `b`. `BL0 add_pair` is unsigned lower-than; `BEQ add_square` handles equality. Otherwise, `a` is greater and the helper finishes.

This source uses unsigned branches because the mathematical quantities and the C interface are unsigned. Do not substitute signed comparisons such as `BLT` without reconsidering the input range.

7. Run and inspect Question 1

Open `Q1_Assembly/sample.uvprojx`, choose the desired target, and build. Start the debugger and run until `tests_done` is 1. You can place a breakpoint after the loop in `Source/sample.c`.

Add `inputs`, `results`, `tests_passed`, and `tests_done` to the Watch window. The C driver calls the assembly on 28, 220, 12496, and 100. Results must be 1, 2, 5, and 0, with `tests_passed == 4`.

For a useful single-step exercise, break at `isSociable` when the argument is 220. At entry `R0` is 220. After the first helper call, `R0` is 284 and `R6` becomes 1. After the second helper call, `R0` is 220 and `R6` becomes 2. Equality with `R4` leads to `found`, which returns 2.

The C driver is a way to call and inspect the answer; it does not replace the required assembly implementation.

8. Question 2: divide responsibilities between main and the handler

The complete application has three relevant source files:

- `Source/sample.c` initializes the board and starts Timer 1.
- `Source/timer/IRQ_timer.c` holds the array and processes one entry per `MR0` interrupt.
- `Source/ASM_funct.s` supplies the assembly calculation.

The template already includes these files in both Keil targets. Its timer source already owns the interrupt handlers. Replacing that source with the supplied version avoids duplicate `TIMER1_IRQHandler` definitions.

The application leaves the calculations inside the interrupt handler because the question explicitly requires the handler to call `isSociable`. The main loop sleeps with `__WFI()` while waiting for interrupts.

9. Timer 1 must interrupt every two seconds

The call in main is:

```
exam_timer_config_ms(EXAM_TIMER1, 2000u, EXAM_TIMER_PERIODIC)
```

The helper configures Timer 1 for 2000 milliseconds. It uses the selected timer's actual peripheral clock, sets the prescaler `PR` to 0, writes match register `MR0`, enables the match interrupt, and resets the timer counter on each match. Configuration stops/resets the timer; the separate `exam_timer_start(EXAM_TIMER1)` call starts it.

The timing relationship is:

```
counter frequency = PCLK / (PR + 1)
period = MR0 * (PR + 1) / PCLK
```

With `PR = 0`, `MR0 = 2 * PCLK` for a two-second interval. For example, if Timer 1's `PCLK` is 25 MHz, `MR0 = 50,000,000`. If it is 100 MHz, `MR0 = 200,000,000`. These are examples, not interchangeable constants; the API reads the clock selection and computes the appropriate count.

For this timer match/reset mechanism, use the interval count itself in `MR0`. Do not automatically subtract one as you might for a `SysTick` reload register.

Periodic `MR0` behavior uses the low three `MCR` bits as `interrupt = 1`, `reset = 1`, `stop = 0`, giving binary 011. The counter resets automatically at each match, so the handler does not manually restart the timer.

The error check in main leaves the program stopped with LEDs off if timer configuration fails. Under the copied template's normal clock settings, the two-second count fits the timer's 32-bit register.

10. Process exactly one array entry per interrupt

The required array is declared as `const uint32_t numbers[7]`. Its contents are exactly the seven unsigned values from the paper. `next_index` initially equals 0 and persists between calls because it has static storage duration.

At the beginning of the handler, `exam_timer_ack(EXAM_TIMER1)` snapshots and clears the active timer flags. The saved value's bit 0 represents `MR0`. If it is not set, this handler returns without consuming an array entry.

Acknowledging early prevents the same pending flag from immediately retriggering the handler. Use the saved snapshot for the test: reading the flags again after clearing them would lose the event. The hardware interrupt register uses write-one-to-clear semantics; the existing API handles that detail.

For an `MR0` event, the handler reads `numbers[next_index]`, calls `isSociable`, and displays its answer. It then increments `next_index`. At 7, it wraps to 0. Valid accesses are always indexes 0 through 6.

The optional `last_input`, `last_result`, and `interrupt_count` variables make debugger observation easy. They are volatile so the debugger can observe actual memory updates. They are not used to choose the next input or alter the mathematical algorithm.

11. Map the answer to the physical LED label

The paper asks for result 1 -> LED4, result 2 -> LED5, through result 8 -> LED11. Therefore:

```
physical LED label = result + 3
```

The supplied API accepts these physical labels, so the handler uses

`exam_led_one_hot((uint8_t)(length + 3u))`. One-hot means exactly one bit is set: the selected LED is on and all the others are off. A zero result calls `exam_led_clear()`.

On this template's board mapping, physical label L corresponds to GPIO P2.(11 - L). Consequently, raw mask bit 7 controls LD4, while bit 0 controls LD11. If using a raw mask instead of the label helper, the correct formula is `1u << (8u - length)` for lengths 1 through 8. Never evaluate that expression for length 0 or an out-of-range length.

Function result	Physical LED	GPIO bit	Raw mask
0	All off	None	0x00
1	LD4	P2.7	0x80
2	LD5	P2.6	0x40
3	LD6	P2.5	0x20
4	LD7	P2.4	0x10
5	LD8	P2.3	0x08
6	LD9	P2.2	0x04
7	LD10	P2.1	0x02
8	LD11	P2.0	0x01

Using `1u << (length - 1u)` would reverse the requested physical LED order. Using an operation that merely turns on the new LED would leave the previous LED on. The one-hot helper solves both requirements.

12. Full expected trace for the seven inputs

8128: return 1

```
8128 -> 8128
```

The first aliquot sum equals the start. It is perfect, so show LD4.

5564: return 2

```
5564 -> 5020 -> 5564
```

It forms an amicable pair with 5020. Show LD5.

5400: return 0

```
5400 -> 13200 -> 32928 -> 67872 -> 137760
      -> 370272 -> 839328 -> 1680672 -> 3568992
```

Eight sums have been computed, none equal to 5400. The final answer is 0, even though the latest sum is not 1. Switch all LEDs off. The program does not compute a ninth term.

14264: return 5

14264 -> 12496 -> 14288 -> 15472 -> 14536 -> 14264

This is the five-term cycle from Question 1, entered at another member. Show LD8.

1305184: return 4

1305184 -> 1264460 -> 1547860 -> 1727636 -> 1305184

Four sums return to the start. Show LD7.

1598470: return 2

1598470 -> 1511930 -> 1598470

Two sums return to the start. Show LD5.

4938136: return 4

4938136 -> 5753864 -> 5504056 -> 5423384 -> 4938136

Four sums return to the start. Show LD7.

Nominal timer schedule

Interrupt	Time from timer start	Input	Result	Display
1	2 s	8128	1	LD4
2	4 s	5564	2	LD5
3	6 s	5400	0	All off
4	8 s	14264	5	LD8
5	10 s	1305184	4	LD7
6	12 s	1598470	2	LD5
7	14 s	4938136	4	LD7
8	16 s	8128	1	LD4 again

These are timer match times. LED updates occur after the corresponding computation finishes. The previous LED state remains during that calculation. The table assumes the handler finishes before the next match. Actual board timing and execution duration have not been measured.

13. Debugging checklist tied to this solution

If the assembly symbol is unresolved, check that `Source/ASM_funcnt.s` is included in the active target and exports `isSociable` with the same capitalization as `C`.

If the linker reports two Timer 1 handlers, use only the supplied definition in `Source/timer/IRQ_timer.c`. Do not paste another one into `sample.c`.

If the timer never fires, verify that main reaches the successful configuration path and the separate start call. Then inspect TCR, MR0, MCR, and the interrupt enable state. Avoid treating simulator wall-clock speed as a physical two-second measurement.

If more than one LED stays lit, ensure the code uses `exam_led_one_hot` or writes a complete replacement mask. If LEDs run in reverse order, distinguish physical LD labels from low-level GPIO indexes.

If the cycle length is one too large, ensure the count starts at zero and is incremented only after an aliquot sum is computed. If an eight-term cycle is rejected, put the original-number comparison before the count limit.

If perfect squares have the wrong aliquot sum, count the square root once. If prime inputs run much longer than expected, remember that the paper's literal algorithm can test every candidate through n ; that behavior is explained in Section 4.

14. What validation does and does not establish

The validation script compiles and links the complete source lists from both project targets with the installed native Arm compiler, assembler, and linker. Logs and `build_results.json` record the actual outcomes. This is command-line native-toolchain validation, not a claim that the Keil GUI build button was used.

It then executes the native compiled instructions in Unicorn. Aliquot sums are compared against an independent square-root-based mathematical reference. Sociable examples include the paper's four examples, all seven array entries, and extra edge cases. The checks also verify preservation of R4-R11 and the stack pointer.

Controlled helper outputs test a successful return on the eighth term, failure after eight nonreturning terms, and immediate termination on 1. These synthetic boundary checks test control flow; they are not presented as discoveries of additional natural-number cycles.

The compiled Timer 1 handler is tested on two complete passes through the real seven-value array. The math routine runs as native ARM instructions; peripheral API calls are mocked. Additional checks cover non-MR0 events and every result-to-LED mapping from 0 through 8. See `validation/results.json` for the saved detailed results.

No physical board has been used. Real interrupt entry, actual GPIO electrical behavior, oscillator accuracy, and worst-case runtime are outside these checks. Successful compilation and emulation do not by themselves establish those hardware properties.

15. Sources and implementation references

The mathematical task, prescribed divisor algorithm, input array, timer interval, and LED requirements come from the user-selected **20230704.pdf**, pages 1 and 2.

Project integration, LED numbering, and timer-helper behavior were checked against the current local **Official Combined Exam API** template, especially `Source/exam_api/exam_api.h`, `Source/exam_api/exam_api.c`, `Source/timer/IRQ_timer.c`, and `sample.uvprojx`. Those declarations and implementations are included in each complete project.

Unsigned division and instruction behavior can be checked in Arm's **Cortex-M3 Devices Generic User Guide**, especially the integer multiply and divide instruction sections:

https://www.keil.com/dd/docs/datashts/arm/cortex_m3/r2p1/dui0552a_cortex_m3_dgug.pdf

Appendix A - Complete assembly (both questions)

File: Q1_Assembly/Source/ASM_funct.s

```

        AREA    |.text|, CODE, READONLY
        PRESERVE8
        THUMB
        EXPORT  isSociable
        EXPORT  aliquotSum

; uint32_t isSociable(uint32_t n)
; R0: input n, then return value (0 or a cycle length from 1 to 8).
isSociable    PROC
        PUSH    {R4-R6, LR}    ; 16 bytes; keep SP 8-byte aligned.
        CMP     R0, #2          ; 0 and 1 are not sociable.
        BLO     not_found
        MOV     R4, R0          ; Preserve original n across BL.
        MOV     R5, R0          ; Current sequence value.
        MOVS    R6, #0          ; Computed sums, excluding input.

sequence_loop
        MOV     R0, R5          ; Argument for aliquotSum.
        BL      aliquotSum      ; R0 becomes s(current).
        ADDS    R6, R6, #1      ; Exactly one new term computed.
        CMP     R0, R4
        BEQ     found           ; Success even on the eighth term.
        CMP     R0, #1
        BLS     not_found       ; 1 per paper; also safely handles 0.
        CMP     R6, #8
        BEQ     not_found       ; No return to n within 8 terms.
        MOV     R5, R0          ; Next input is the latest sum.
        B       sequence_loop

found
        MOV     R0, R6          ; Return cycle length.
        POP     {R4-R6, PC}

not_found
        MOVS    R0, #0
        POP     {R4-R6, PC}
        ENDP

```

Appendix A (continued) - Aliquot-sum helper

```

; uint32_t aliquotSum(uint32_t n)
; Implements the paper's divisor-pair algorithm literally for n >= 2.
; R0=n, R1=sum, R2=a, R3=b, R12=remainder. No nested calls.
; Exam inputs and intermediate sums are assumed to fit uint32_t.
aliquotSum    PROC
                CMP     R0, #2
                BLO     small_number
                MOVS     R1, #1          ; Include divisor 1, exclude n.
                MOVS     R2, #2          ; First candidate divisor.
divisor_loop
                UDIV     R3, R0, R2      ; b = floor(n / a).
                MLS     R12, R3, R2, R0 ; remainder = n - b*a.
                CMP     R12, #0
                BNE     next_divisor    ; Not divisible: try a+1.
                CMP     R2, R3
                BLO     add_pair         ; a < b: add two distinct divisors.
                BEQ     add_square       ; a == b: count sqrt(n) once.
                B        sum_done        ; a > b: pair already counted.
add_pair
                ADDS     R1, R1, R2
                ADDS     R1, R1, R3
                B        next_divisor
add_square
                ADDS     R1, R1, R2
sum_done
                MOV     R0, R1
                BX      LR
next_divisor
                ADDS     R2, R2, #1
                B        divisor_loop
small_number
                MOVS     R0, #0
                BX      LR
                ENDP
                ALIGN
                END

```

Appendix B - Question 1 test driver

File: Q1_Assembly/Source/sample.c

```
#include "LPC17xx.h"
#include "exam_api.h"

extern uint32_t isSociable(uint32_t n);

/* Inspect results and tests_passed in the debugger after tests_done = 1. */
const uint32_t inputs[4] = {28u, 220u, 12496u, 100u};
const uint32_t expected[4] = {1u, 2u, 5u, 0u};
volatile uint32_t results[4];
volatile uint32_t tests_passed = 0u;
volatile uint32_t tests_done = 0u;

int main(void)
{
    uint32_t i;
    exam_init();
    for (i = 0u; i < 4u; ++i) {
        results[i] = isSociable(inputs[i]);
        if (results[i] == expected[i]) {
            ++tests_passed;
        }
    }
    tests_done = 1u;
    while (1) {
        __WFI();
    }
}
```

Appendix C - Question 2 main program

File: Q2_Timer_LEDs/Source/sample.c

```
#include "LPC17xx.h"
#include "exam_api.h"

/* Timer 1's handler and the array are in Source/timer/IRQ_timer.c. */
int main(void)
{
    exam_init();
    exam_led_clear();
    if (exam_timer_config_ms(EXAM_TIMER1, 2000u,
                             EXAM_TIMER_PERIODIC) != EXAM_OK) {
        while (1) { __WFI(); }
    }
    exam_timer_start(EXAM_TIMER1);
    while (1) {
        __WFI();
    }
}
```

Appendix D - Question 2 complete timer handlers

File: Q2_Timer_LEDs/Source/timer/IRQ_timer.c

```
#include "LPC17xx.h"
#include "exam_api.h"

extern uint32_t isSociable(uint32_t n);

const uint32_t numbers[7] = {
    8128u, 5564u, 5400u, 14264u, 1305184u, 1598470u, 4938136u
};
static uint32_t next_index = 0u;

/* Optional debugger observations; they do not control the algorithm. */
volatile uint32_t last_input = 0u;
volatile uint32_t last_result = 0u;
volatile uint32_t interrupt_count = 0u;

void TIMER1_IRQHandler(void)
{
    uint32_t pending = exam_timer_ack(EXAM_TIMER1);
    uint32_t length;

    if ((pending & 1u) == 0u) {
        return; /* Only MR0 advances the array. */
    }
    last_input = numbers[next_index];
    length = isSociable(last_input);
    last_result = length;
    ++interrupt_count;

    if ((length >= 1u) && (length <= 8u)) {
        /* Physical board label: 1 -> LD4, 2 -> LD5, ..., 8 -> LD11. */
        (void)exam_led_one_hot((uint8_t)(length + 3u));
    } else {
        exam_led_clear();
    }

    ++next_index;
    if (next_index == 7u) {
        next_index = 0u;
    }
}

/* The template owns one handler per timer. Unused timers stay stopped. */
void TIMER0_IRQHandler(void) { (void)exam_timer_ack(EXAM_TIMER0); }
void TIMER2_IRQHandler(void) { (void)exam_timer_ack(EXAM_TIMER2); }
void TIMER3_IRQHandler(void) { (void)exam_timer_ack(EXAM_TIMER3); }
```